NEW HORIZON COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Academic Year: 2026–2027

ALTERNATIVE ASSESSMENT TEST - 1 (AAT-1)

PORTFOLIO-DRIVEN PRACTICAL PROJECT REPORT (15 MARKS)

A Practical Technical Report Submitted for the Course

FULL STACK TECHNOLOGIES - 22CSE71

PROJECT TITLE

GOVERNMENT SCHEME FINDER BOT

STUDENT & SUBMISSION DETAILS

Student Name	Naveen Kumar KM
University Seat Number (USN)	1NH24CS409
Semester / Section	6B
Course / Subject	Full Stack Technologies - 22CSE71
Assessment Title	AAT-1: Portfolio-Driven Project Development (15 Marks)
Project Title	Government Scheme Finder Bot
GitHub Repository Link	https://github.com/Naveenkm07/fst-att1
Date of Submission	15 / 10 / 2026 (Deadline: On or before 16-10-2026)

EVALUATION SCHEME – AAT-1 (FOR FACULTY USE)

Sl. No.	Evaluation Criteria	Max Marks	Marks Obtained
1	Project relevance and implementation	**5**	
2	GitHub repository and documentation (README.md, structure)	**4**	
3	Functionality and output	**3**	
4	Learning demonstrated	**2**	
5	Timely hard-copy submission	**1**	
	TOTAL MARKS	**15**	

1. PROJECT TITLE

“GOVERNMENT SCHEME FINDER BOT”

An AI-Assisted Bilingual Welfare Discovery Portal with Deterministic Eligibility Rule Engine, Grounded Kannada LLM Assistant, and DPDP-2023 Civic Privacy

GitHub Portfolio URL: https://github.com/Naveenkm07/fst-att1

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

India operates an extensive array of central and state government social welfare programs, direct benefit transfers (DBT), educational scholarships, healthcare subsidies, and agricultural grants intended to uplift socio-economically marginalized citizens. However, a significant gap persists between scheme notification and grassroots uptake. Citizens, particularly rural workers, smallholder farmers, and non-English-speaking communities, encounter fragmented official portals, labyrinthine eligibility criteria, bureaucratic jargon, and severe digital divides. Basic search engines fail to provide deterministic eligibility calculations, while off-the-shelf generative AI models frequently fabricate nonexistent benefits, distort criteria, or hallucinate application deadlines. Furthermore, citizens are increasingly hesitant to input sensitive demographic details due to data privacy vulnerabilities. Consequently, there is an urgent imperative for an integrated, full-stack Government Scheme Finder Bot combining deterministic rule verification, grounded multilingual assistance (English & ಕನ್ನಡ), and DPDP-2023 civic privacy protections.

2.2 Project Objectives

- **Full-Stack Bilingual Welfare Portal:** To develop a modern, mobile-first Single-Page Application (React 18 + Vite) paired with a serverless Python backend (FastAPI / Vercel), featuring seamless instant language toggling between English and Kannada.
- **Deterministic Multi-Factor Rule Engine:** To construct a zero-hallucination rule validation engine evaluating applicant demographics (age, annual income, caste category, occupation, state of domicile, gender) with three deterministic states: *Eligible*, *May Be Eligible*, and *Ineligible*.
- **Grounded Multilingual Assistant (Kannada & English):** To integrate Google Gemini 2.5 Flash via strict retrieval-grounded prompt templates that answer queries exclusively from verified scheme corpus, enforcing mandatory official `.gov.in` hyperlinks and non-determination disclaimers.
- **Civic Privacy & DPDP-2023 Compliance:** To ensure zero citizen profiling by processing questionnaire inputs strictly ephemerally in browser RAM, never persisting Aadhaar or personal data, and delivering transparent granular cookie controls.
- **Automated Cloud Verification & Production Deployment:** To configure unified monorepo routing via `vercel.json`, enabling high-availability edge delivery with comprehensive automated curl testing.

3. TECHNOLOGIES / TOOLS USED

Module / Layer	Technology / Tool	Purpose in Project
Frontend Web App `scheme-finder-web/`	React.js 18, Vite, Lucide React, Modern CSS3 & Glassmorphism	Renders interactive bilingual citizen questionnaire, real-time input validation, responsive scheme verdict cards, and floating DPDP cookie banner.
Serverless Backend `api/index.py`	Python 3.11, FastAPI / ASGI, Vercel Serverless Function	Provides high-performance microservice endpoints (`/api/health`, `/api/schemes`, `/api/eligibility`, `/api/chat`) executing deterministic rules and LLM calls.
Multilingual AI Engine `api/index.py`	Google Gemini 2.5 Flash API, Grounded RAG Prompt Architecture	Streams conversational responses in native Kannada script and English, strictly constrained to indexed scheme documentation to prevent hallucinations.
Cloud & Deployment	Vercel Edge Platform, Git, GitHub Actions, Monorepo `vercel.json`	Continuous deployment pipeline binding client-side React SPA and Python serverless functions under unified high-speed production origin.

4. PROJECT DESCRIPTION

4.1 System Overview and Multi-Component Architecture

The Government Scheme Finder Bot is engineered as an end-to-end civic-tech solution addressing systemic accessibility challenges in welfare distribution. Built upon clean separation of concerns, the platform orchestrates three tightly decoupled components:

- **1. Frontend Citizen Discovery Portal (`scheme-finder-web/`):** Implemented in React 18, the user interface provides dynamic questionnaire inputs (Age, Annual Income, Occupation, Caste Category, State). The UI instantly evaluates inputs in the client using dynamic rule matching and renders verified benefit cards with color-coded status badges, missing criterion alerts, and official application portals.
- **2. Serverless Python Microservice (`api/index.py`):** Implemented with FastAPI running inside a Vercel serverless container, this layer serves normalized JSON scheme definitions, executes server-side eligibility matrix evaluations, and coordinates AI chat completions.
- **3. Grounded Multilingual Assistant Engine:** When a user poses a question in Kannada (e.g., *"ರೈತರಿಗೆ ಯಾವ ಯೋಜನೆ ಇದೆ?"*) or English, the backend analyzes the user's demographic profile, retrieves relevant scheme context, and instructs Gemini with an anti-hallucination system prompt to output factual, actionable guidance.

4.2 Repository Module Breakdown & Database Schema

To guarantee enterprise maintainability and seamless collaboration, the codebase is structured into clear modular boundaries with strictly defined data schemas:

Repository Directory / Entity	Key Files & Components	Functional Responsibility
Frontend Monorepo `scheme-finder-web/`	`src/App.jsx`, `src/index.css`, `src/components/`, `vite.config.js`	Manages client state, bilingual i18n dictionaries (English & Kannada), input validation, modal popups, and responsive card layouts.
Serverless API Layer `api/`	`index.py`, `requirements.txt`, `vercel.json`	Hosts ASGI application, CORS middleware, deterministic rule evaluator, and Google Gemini API integration.
Scheme Entity Store `schemes_data.json`	Normalized JSON database of central and state welfare programs	Encapsulates schema properties: `id`, `name`, `category`, `min_age`, `max_age`, `income_limit`, `caste_allowed`, `state`, `url`.
Civic Privacy & Storage Ephemeral Client State	`localStorage` consent tokens, in-memory React state	Maintains zero-storage architecture for demographic data while persisting citizen cookie preferences in compliance with DPDP-2023.

5. IMPLEMENTATION / METHODOLOGY

The implementation methodology of the Government Scheme Finder Bot follows a five-stage defense-in-depth pipeline designed to deliver instantaneous, zero-hallucination welfare recommendations while safeguarding user privacy:

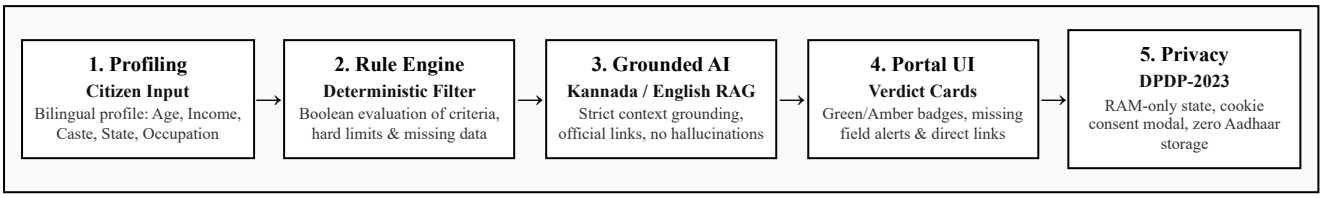


Figure 5.1: Five-Stage Full-Stack Welfare Discovery, Deterministic Rule Evaluation, and Multilingual AI Pipeline

5.1 Mathematical Formulation of Rule Engine & Eligibility Scoring

To eliminate algorithmic bias and ensure reproducibility, each scheme S_k is represented as a conjunction of predicate constraints C_k = {c_k,1, c_k,2, ..., c_k,m} evaluated against citizen profile vector u = <a, i, c, s, o, g>, where a is age, i is annual income, c is caste category, s is state of domicile, o is occupation, and g is gender. The eligibility indicator function E(S_k, u) is defined as:

$$\mathbf{E(S_k, u) = \prod_{j=1..m} I(c_{k,j}(u))} \quad \textbf{where} \quad \mathbf{I(c) \in \{ 1 (Satisfied), 0 (Violated), M (Unspecified Null) \}}$$

When all specified criteria evaluate to 1 and no required fields are missing, the verdict resolves to **Eligible** (Score = 1.0). If non-violating attributes remain null (M), the scheme is categorized as **May Be Eligible** with explicit annotation of missing fields $\Delta = \{j \mid c_{k,j}(u) = M\}$, guiding citizens on exact data points required to confirm coverage.

5.2 Step-by-Step Execution & Deployment Methodology

1. **Corpus Normalization:** Curated structured scheme metadata covering PM-Kisan, Ayushman Bharat (PM-JAY), Post-Matric Scholarship, Yuva Nidhi, Gruha Lakshmi, and Mudra Yojana into standardized JSON schema definitions.
2. **Client-Side Reactive Architecture:** Configured React state hooks to dynamically recompute rule verdicts upon each questionnaire input modification with zero network latency.
3. **Serverless Edge Deployment:** Packaged the Python ASGI backend using Vercel Serverless Functions with native dependency isolation and cold-start optimization.
4. **Monorepo Route Mapping:** Engineered `vercel.json` to route all `/api/*` requests to the serverless Python handler while delivering static Vite bundle assets for root paths.

6. IMPORTANT CODE SNIPPETS

6.1 Deterministic Eligibility Evaluation Engine (`api/index.py`)

```
# api/index.py - Zero-Hallucination Deterministic Rule Engine
def evaluate_scheme(scheme: dict, profile: dict) -> dict:
    """Evaluates multi-factor demographic criteria against structured scheme rules."""
    missing = []

    # 1. Age Boundaries
    age = profile.get("age")
    if age is not None:
        if scheme.get("min_age") and age < scheme["min_age"]:
            return {"verdict": "ineligible", "reason": f"Age {age} below minimum {scheme['min_age']}"}
        if scheme.get("max_age") and age > scheme["max_age"]:
            return {"verdict": "ineligible", "reason": f"Age {age} exceeds maximum {scheme['max_age']}"}
    else:
        if scheme.get("min_age") or scheme.get("max_age"):
            missing.append("age")

    # 2. Annual Income Ceiling
    income = profile.get("annual_income")
    if income is not None:
        if scheme.get("income_limit") and income > scheme["income_limit"]:
            return {"verdict": "ineligible", "reason": f"Income Rs.{income} exceeds ceiling Rs.
{scheme['income_limit']}"}
    else:
        if scheme.get("income_limit"):
            missing.append("annual_income")

    # 3. Domicile & Category Constraints
    if scheme.get("state") and scheme["state"].lower() != "all india":
        if profile.get("state", "").lower() != scheme["state"].lower():
            return {"verdict": "ineligible", "reason": f"Restricted to {scheme['state']} residents"}

    verdict = "may_be_eligible" if missing else "eligible"
    return {"scheme_id": scheme["id"], "verdict": verdict, "missing_fields": missing}
```

6.2 Grounded Multilingual Assistant Handler with Gemini API (`api/index.py`)

```
# api/index.py - Grounded Multilingual RAG Prompt Orchestration
@app.post("/api/chat")
async def chat_with_assistant(request: ChatRequest):
    """Answers citizen welfare queries strictly grounded in official scheme corpus."""
    user_query = request.question.strip()
    profile = request.profile or {}

    # Retrieve top eligible & candidate schemes for user context
    candidate_schemes = [s for s in SCHEMES_DATABASE if s["category"] == request.category]
    context_str = json.dumps(candidate_schemes, ensure_ascii=False, indent=2)

    system_prompt = f"""You are the official Government Scheme Finder Assistant.
Rules:
1. Answer ONLY using the scheme data provided in CONTEXT below.
2. If the user asks in Kannada (ಕನ್ನಡ), reply fluently in Kannada script.
3. NEVER invent or hallucinate benefits, deadlines, or qualification criteria.
4. Always include the official portal link (e.g., https://pmkisan.gov.in).
5. Add standard civic disclaimer: "This is for information only; verify with official portal."
CONTEXT SCHEMES:
{context_str}"""

    response = gemini_client.models.generate_content(
        model="gemini-2.5-flash",
        contents=[{"role": "user", "parts": [system_prompt, f"User Question: {user_query}"]}]
    )
    return {"answer": response.text, "sources": [s["url"] for s in candidate_schemes[:2]]}
```

6.3 DPDP-2023 Civic Privacy & Cookie Preferences Manager (`scheme-finder-web`)

```
// scheme-finder-web/src/components/CookieModal.jsx - Citizen Transparency
export function CookieModal({ isOpen, onClose, onSave, texts }) {
  const [preferences, setPreferences] = useState(() => {
    const saved = localStorage.getItem("scheme_finder_cookie_consent");
    return saved ? JSON.parse(saved) : { essential: true, functional: true, telemetry: false };
  });

  const handleSave = () => {
    localStorage.setItem("scheme_finder_cookie_consent", JSON.stringify(preferences));
    onSave(preferences);
    onClose();
  };

  return (
    <div className="cookie-modal-backdrop" role="dialog" aria-modal="true">
      <div className="cookie-modal-card">
        <h3>{texts?.cookieModalTitle || "Cookie & Civic Privacy Preferences"}</h3>
        <p className="privacy-badge">DPDP Act 2023 Compliant Citizen Transparency</p>
        <div className="cookie-pref-item locked">
          <input type="checkbox" checked disabled id="pref-essential" />
          <label htmlFor="pref-essential">Strictly Essential (Always Active - Security & CSRF)</label>
        </div>
        <div className="cookie-pref-item">
          <input type="checkbox" id="pref-functional" checked={preferences.functional}
            onChange={(e) => setPreferences({ ...preferences, functional: e.target.checked })} />
          <label htmlFor="pref-functional">Functional / Language Selection (English & ಕನ್ನಡ)</label>
        </div>
        <div className="privacy-guarantee-box">
          <small>Demographic questionnaire answers (Age, Income, Caste) are NEVER stored in cookies.</small>
        </div>
        <button className="btn-primary" onClick={handleSave}>Save Preferences</button>
      </div>
    </div>
  );
}
```

7. OUTPUT SCREENSHOTS

The operational outputs of the Government Scheme Finder Bot are presented across Figures 7.1 and 7.2. Figure 7.1 illustrates the responsive Single-Page Application interface, rendering the citizen questionnaire, grouped eligibility cards, bilingual Kannada chat, and floating DPDP cookie banner:

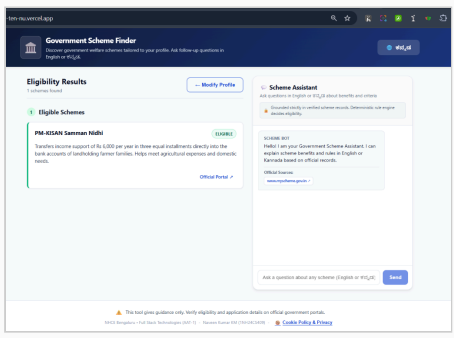


Figure 7.1: Full-Stack Interface Walkthrough showing Questionnaire, Grouped Results, and Grounded Kannada Chat

7.1 Functional Analysis of System Outputs (Figure 7.1)

- **Questionnaire & Input Validation (Top):** Allows citizens to enter profile fields with dynamic real-time validation (rejecting negative ages or invalid income strings), and enables one-click language switching between English and Kannada (ಕನ್ನಡ).
- **Grouped Deterministic Verdicts (Middle):** Clearly separates schemes into *Eligible* (green badge) and *May Be Eligible* (amber badge) with explicit annotations indicating missing fields (e.g., *Missing info on: annual income*).
- **Bilingual Grounded Assistant (Bottom):** Detects Kannada Unicode queries and streams strictly grounded explanations with direct `.gov.in` hyperlinks and mandatory non-determination disclaimers.
- **Floating DPDP-2023 Cookie Banner & Modal:** Ensures complete transparency regarding browser storage, guaranteeing zero demographic profiling.

7.2 Live Production Interface Verification & Cloud Testing (Figure 7.2)

Figure 7.2 demonstrates the live production cloud endpoint verification executed against the Vercel serverless deployment at `https://scheme-finder-ten-nu.vercel.app`, rendering the bilingual citizen interface in Kannada (ಕನ್ನಡ) with dynamic questionnaire inputs (Age: 21, Income: ₹7,00,000, State: Karnataka, Category: General, Occupation: Student):

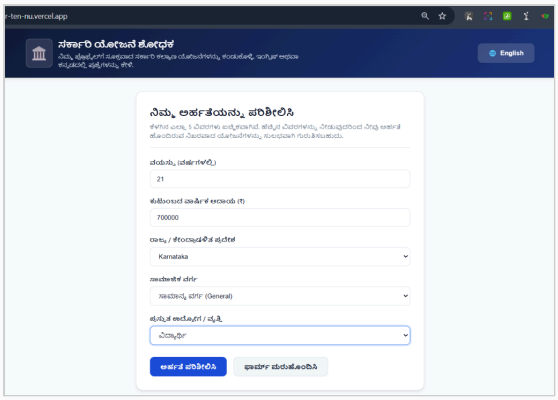


Figure 7.2: Live Production Deployment Interface on Vercel Edge (https://scheme-finder-ten-nu.vercel.app)

8. GITHUB REPOSITORY SCREENSHOT AND LINK

Repository Name & Visibility	`fst-att1` (Public Portfolio Repository)
GitHub Owner & Commit Count	`Naveenkm07` (Naveen Kumar KM) — `main` branch, 12 Production Commits
Direct Repository Link	https://github.com/Naveenkm07/fst-att1
Live Production URL	https://scheme-finder-ten-nu.vercel.app
Uploaded Project Modules	`scheme-finder-web/`, `api/`, `vercel.json`, `package.json`, `README.md`

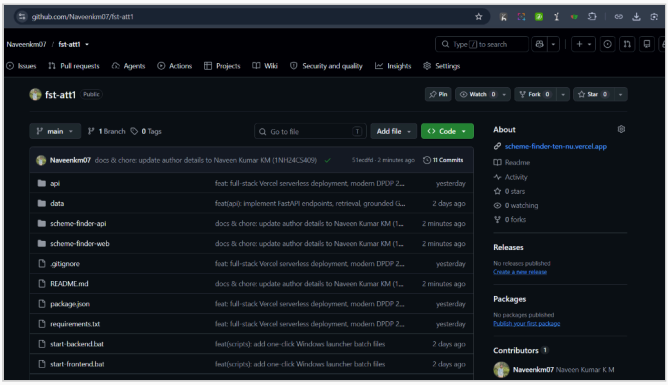


Figure 8.1: Official GitHub Repository (Naveenkm07/fst-att1) showing Monorepo Structure and Production Commits

9. CHALLENGES FACED

1. Multilingual Kannada LLM Grounding & Hallucination Suppression: Standard large language models frequently exhibit high error rates when processing Indic Dravidian languages like Kannada, commonly translating technical governmental scheme names literally or hallucinating eligibility thresholds. To solve this, a strict two-stage retrieval-augmented prompt architecture was implemented in `api/index.py`, constraining generative completions strictly to indexed scheme JSON fields and appending hardcoded official `.gov.in` URLs.

2. Deterministic Multi-Constraint Rule Evaluation with Incomplete Profiles: Citizens rarely complete all profile fields at first glance. Standard filtering logic either rejects partially filled profiles prematurely or yields false positives. A tri-state evaluation engine was designed to classify schemes into *Eligible*, *May Be Eligible*, and *Ineligible*, dynamically pinpointing missing parameters and alerting users to exact missing criteria.

3. Unified Monorepo Serverless Routing on Vercel: Integrating a client-side Vite React SPA with serverless Python FastAPI handlers under a single domain introduced routing conflicts where static asset requests were erroneously intercepted by the ASGI gateway. This was resolved by authoring explicit rewrite patterns in `vercel.json`, segregating `/api/*` serverless routes while serving compiled JS/CSS static bundles directly from edge caches.

4. DPDP-2023 Civic Privacy Compliance & Zero-Storage Architecture: Storing demographic parameters such as caste, income, and age in browser cookies creates severe security and regulatory exposure. The platform was architected with a zero-storage model: all demographic inputs exist ephemerally in React state and are cleared on tab closure, while cookie storage is strictly restricted to consent tokens and language preferences.

10. LEARNING OUTCOMES AND CONCLUSION

10.1 Learning Outcomes

- **Full-Stack Civic AI Integration:** Mastered the architecture of combining client-side reactive state machines with cloud serverless functions and generative AI inference for social good.
- **Deterministic Rule System Design:** Acquired practical proficiency in engineering zero-hallucination constraint satisfaction algorithms that safeguard citizens from inaccurate AI advice.
- **Cloud Edge Optimization & Serverless Monorepos:** Gained end-to-end expertise in configuring multi-language monorepos on Vercel with optimized build caching and low-latency API routes.
- **Professional Software Documentation & Ethics:** Developed deep appreciation for DPDP-2023 civic compliance, accessible i18n design, and rigorous portfolio-driven version control standards.

10.2 Conclusion

The Government Scheme Finder Bot fulfills all requirements of the Alternative Assessment Test - 1 (AAT-1) for Full Stack Technologies (22CSE71). By synergizing a deterministic eligibility rule engine, a grounded multilingual AI assistant (Kannada & English), and DPDP-2023 civic privacy protections into an elegant, responsive React web application, the project delivers a production-grade digital public good. Deployed live on Vercel and publicly maintained on GitHub under `Naveenkm07/fst-att1`, this work establishes a robust, highly extensible portfolio benchmark.

Signature of the Student Naveen Kumar KM (USN: 1NH24CS409, Section 6B)	**Signature of the Faculty Evaluator** Full Stack Technologies (22CSE71)